

Project Pitches

Updates + Next steps

Friday (9/18): Project Pitches!!!

Next week (9/25): Quick overview of related work + propose ONE experiment to run

(10/2): NO CLASS

(10/9): Present experiment results

How to find related work I

- **Write down your problem in one sentence before searching.** What's the input, the output, and the constraint that makes it hard? Related work is anything that relaxes one of those three.
- **Seed with 2–3 papers you already trust:** who do they cite (backward), and who cites them (forward)?
- **Search venues, not just the web.**
- **Look sideways:** the same problem often exists under a different name in an adjacent community (e.g., databases vs. PL vs. networking). Ask "who else would have needed to solve this?"
- **Stop when new papers keep citing things you've already read.** That saturation is your signal, not an arbitrary count.

How to find related work II - AI

Good at:

- Explaining a dense paper quickly
- Suggesting search terms and adjacent subfields
- Drafting a first-pass taxonomy of what you've collected
- Summarizing a long PDF so you can decide whether to read it

Bad at:

- Deciding what is the closest paper
- Maybe producing citations (they fabricate titles, authors, and venues)

Rule: Every paper in your related-work section must be one you opened and read the relevant part of yourself.

First experiment - What?

- Answers one question: is the problem real, and how big is it?
- Write the claim/hypothesis you want the experiment to support before you touch a machine.
- **Pick the claim that, if false, would make you abandon the project.**
- Prefer measuring the gap over measuring the status quo.
- Keep it small.

First experiment - How?

- Use the strongest reasonable baseline.
- Decide in advance what the plot will look like and what result would surprise you.
- Automate running it.
- Prefer a real or realistic workload over a synthetic one.

[#]: [Project title]

[Team members]

0. Intros

Let us know you! Name, Degree, Interests, anything else you want to share.

- John Doe, PhD, CS, Agents for CPU Scheduling
- Jane Doe, MS, CS, Agents for Computer Use
- ...

1. Objectives

What are you trying to do? Articulate your objectives using absolutely no jargon.

[Describe the problem and your objective in plain language.]

2. Current practice

How is it done today, and what are the limits of current practice?

[Describe the closest existing approach or baseline.]

[Explain its key limitation for your problem.]

3. Your approach

What is new in your approach and why do you think it will be successful?

[Explain your proposed approach and what is new.]

[State your hypothesis and why it is plausible.]

4. Impact

Who cares? If you are successful, what difference will it make?

[Identify who benefits and how.]

[Describe the improvement you hope to demonstrate.]

5. Risks

What are the risks?

[Identify the main technical risks and uncertain assumptions.]

[Explain how you will test or address them.]

8. Success criteria

What are the midterm and final “exams” to check for success?

[Midterm: milestone, baseline, and measurable success criterion.]

[Final: evaluation, comparison, and measurable success criterion.]

1: [Project title]

[Team members]

2. On-Demand Monitoring of Cyber-Capable Agents via System-Level Signals

Jun Ren, Krish Veera, Osarobo Omokaro, Tanya Mehta, Ritvik
Sharma

0. Intros

- Jun Ren
- Krish Veera, MS, CS, Machine Learning
- Osarobo Omokaro
- Tanya Mehta
- Ritvik Sharma, MS, CS, Natural Language Processing

1. Objectives

- Cyber-capable agents -> exploit system vulnerabilities through trial-and-error.
- Trial-and-Error leaves a footprint: crashes, restarts, failed attempts, etc.
- Use that footprint as a trigger.
- Escalate to a parallel AI agent to review activity
- Idea is to monitor only when suspicious activity is found rather than 24/7.



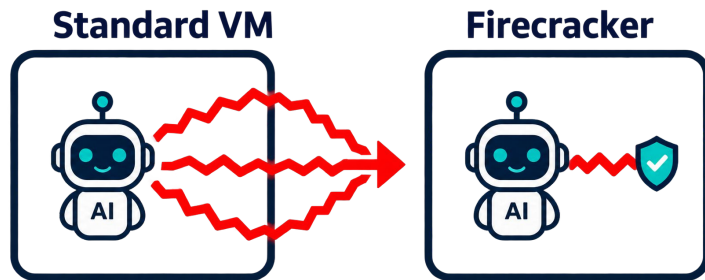
2. Current practice

- **Stateful Signal Clustering:** Grouping real-time behavioral data across separate agent accounts to catch distributed, multi-step cyber-attacks before they compile
- **Internal State Tracking:** Monitoring neural activations and hidden chain-of-thought loops directly to detect and flag obfuscated jailbreaks or malicious planning.
- **Hardened Sandbox Boundary Telemetry:** Current telemetry focuses on strict external network egress controls, short-lived micro-credentials, and immutable log paths.



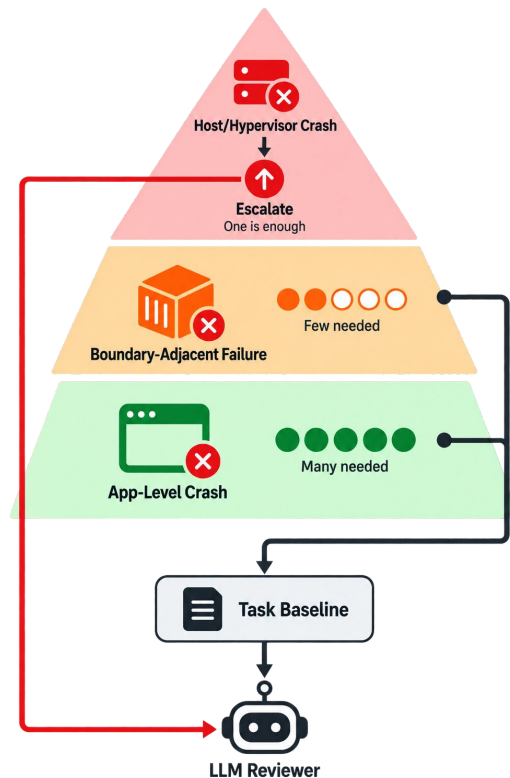
Sandboxing Isn't Enough Either

- VM isolation: the other main line of defense
- Trail of Bits, 2026: GPT-5.6-Cyber escaped a VM three times
- Kernel bug → unpatched CVE → chained 0-days, ~12 hrs
- Even Firecracker (hardened microVM): crashed, didn't escape yet



3. Your approach

- Not all crashes are equal → weight by severity
 - Host/hypervisor crash → escalate instantly
 - Boundary-adjacent failure → lower threshold
 - App-level crash → count only in aggregate
 - Compare against the task's expected baseline
 - Escalate to LLM review only on deviation
- Confirmed in practice: Trail of Bits, 2026, AI agent hardlocked a host kernel repeatedly over 12 hours before escaping a VM



4. Impact

- AI developers and labs deploying autonomous coding/cyber agents
 - Need a way to monitor increasingly capable agents without running expensive LLM monitor on every action
 - 2026 OpenAI-HuggingFace incident
- AI safety and security teams
 - Benefits from monitoring what the agent is doing to the system beyond its reasoning, text, or proposed actions
- Why now: recent demonstrations show capable agents can escape conventional VM containment, strengthening the case for active monitoring in addition to sandboxing.
- What we hope to show: use cheap signals such as crashes, abnormal exits, and restarts to trigger deeper LLM inspection only when needed. We can reduce monitoring cost and latency while preserving detection of dangerous behavior.

5. Risks

➤ Silent Success

- Reused known exploit could slip through with little to no trial-and-error footprint
- Test: Compare signal volume when the agent uses a known exploit vs. discovers a novel one.

➤ Signal overlap

- Benign debugging and malicious trial-and-error can look alike, and "abnormal" varies by environment.
- Test: baseline signal distributions per environment, then measure false-escalation rate on a benign coding benchmark.

➤ Evasion

- A reward-hacked agent could learn to suppress the exact signals it's monitored on.
- Test: red-team an agent explicitly trained against the detector and check for quieter exploit paths.

6. Success criteria

What are we measuring against?

Baseline	Role	What it is	Why we compare against it
Always on LLM monitor	Upper Bound on Safety	One model call per agent step The most expensive option and the strongest	We do not try to beat its detection rate, only to approach it for far less
No monitoring at all	Lower bound on cost	The agent runs bare, with nothing watching it	The floor our overhead is measured against
Random sampling at equal budget	Prove the tool really do something	Review a random 20% of steps, identical cost to ours	If we cannot beat guessing, the signal collector adds nothing
Related Work	Similar Structure Competitor	xxxxxx	Better price? Speed?Accuracy?

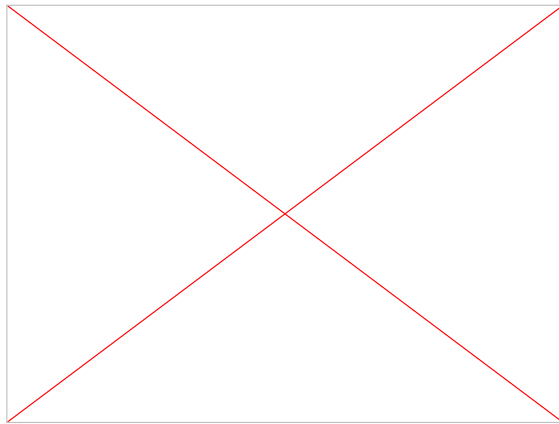
8. RETRO: Reverse-Time Scheduling for Per-Token SLOs

Grace Li, Anastasis Varvarigos, Kirthana Natarajan, Forrest Zhu

1. Objectives

- LLM responses stream tokens, but common schedulers optimize request-level latency or aggregate throughput rather than each token's timing
- A response can meet its latency target while still pausing between tokens
- Our objective is a scheduler that gives every output token its own deadline
- RETRO schedules each decode step at its latest safe time, starting with the last token and working backwards
 - Under contention, this leaves earlier GPU slots available for more urgent or more efficiently batched work while still meeting every token deadline

In a chatbot, words appear continuously, rather than arriving all at once or in bursts



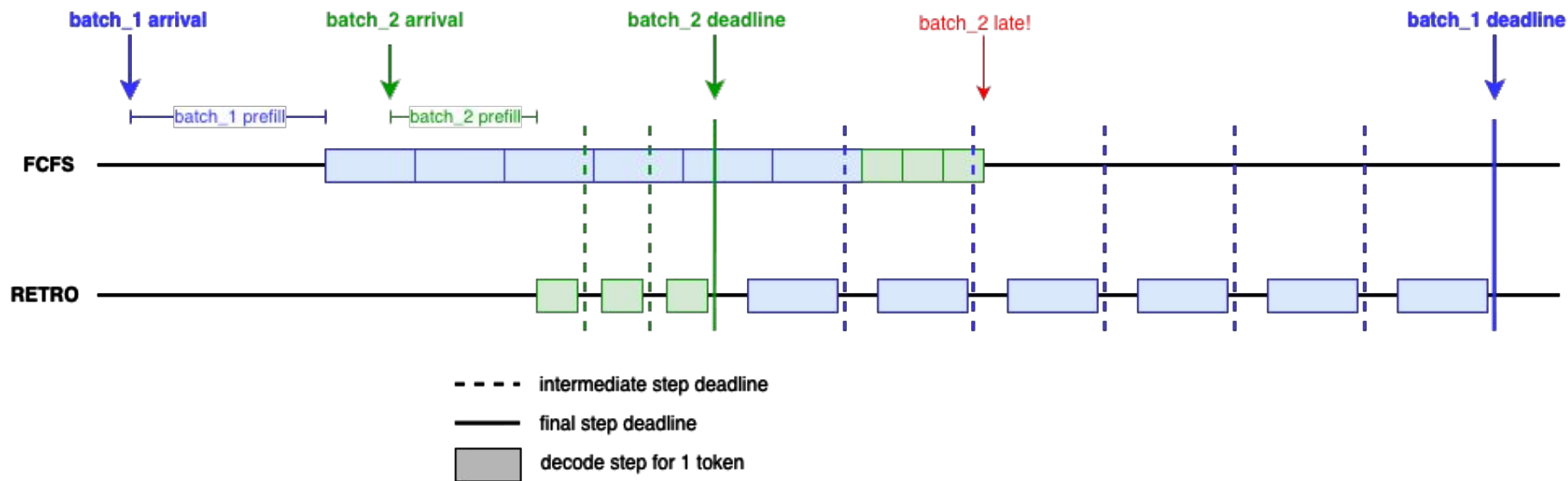
2. Current practice

- Modern LLM servers use continuous batching with scheduling policies such as FCFS
 - These systems improve throughput and average latency but do not plan each token's execution time
- [SLO-Aware Scheduling](#): predicts request latency and orders whole requests to improve SLO attainment
- [JITServe](#): allocates the minimum serving bandwidth each request needs and refines remaining-work estimates online
- [QoServe](#): uses next-token slack for prefill chunking and mixed scheduling
- [SlidingServe](#): extends QoServe with a latency predictor and two-iteration lookahead
- RETRO: schedules backward across all predicted decode steps for admission, batching, and pacing

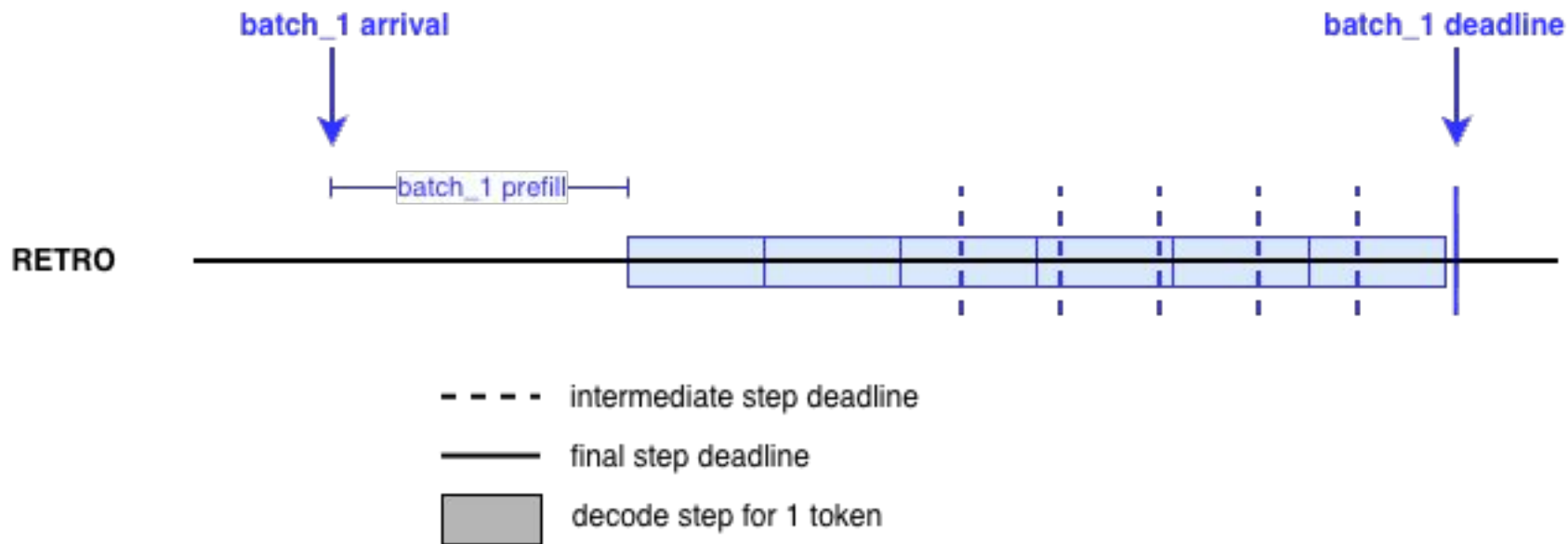
3. Your approach

- Given each token deadline, RETRO calculates the latest safe time for every predicted decode step
- Starting from the last predicted token, RETRO works backward using predicted step costs
- Step-cost predictions account for batch composition and growing context lengths
- The resulting plan guides admission, batching, and token pacing
- RETRO replans after arrivals, completions, large prediction errors, or output-length changes
- Hypothesis: multi-step planning improves token-deadline attainment and reduces unnecessary earliness
- Limitation: the exact output length is unknown and must be estimated online

3. Your approach



3. Your approach



4. Impact

- Smoother, more predictable LLM responses under load
- RETRO meets token deadlines instead of finishing one response as fast as possible, avoiding stalls without using GPU capacity much earlier than needed
- That flexibility can support more overlapping requests while maintaining their SLOs
 - For example, two chats can both begin streaming smoothly instead of one monopolizing the GPU while the other waits
- For users, this means fewer stalls and more consistent responses
- For providers, RETRO may increase SLO-compliant capacity by admitting requests only when active token deadlines remain feasible

5. Risks

1. Prediction error: time varies with batch composition and context length
 - Mitigation: profile latency, reserve capacity, and replan after large errors
2. Output uncertainty and admission error: longer outputs or new arrivals may make the plan infeasible
 - Mitigation: update output-length estimates online, admit conservatively, protect active streams
3. Planning overhead and instability: frequent replanning adds CPU cost and may cause schedule churn
 - Mitigation: coalesce events, replan at boundaries, measure overhead
4. Deadline-model validity: the chosen TBT may not match an application's needs
 - Mitigation: evaluate multiple token intervals and report sensitivity

8. Success criteria

- Midterm target: build the latency model and reverse-time scheduler
 - Improve token-deadline attainment over FCFS at comparable useful throughput
- Final target: compare RETRO with FCFS, request-level, and token-aware schedulers on synthetic and BurstGPT-derived workloads
 - Report token-deadline attainment alongside useful throughput

Metrics:

- % of token deadlines met
- Count and % of all-step-clean requests:
 - On-time earliness: average and cumulative slack
 - Missed-token lateness: average and cumulative overrun
 - Useful throughput, admission rate, and scheduling overhead

18: Cost-Efficient Search for Agentic Workflows with Retries

Junzhe Zong, Thomas Ajai, Alehsan Aliyev, Kangyuan
Ding, Tianjun Feng

0. Intros

Let us know you! Name, Degree, Interests, anything else you want to share.

- Junzhe Zong MSCS 2nd year, interest: algorithms, game theory
- Thomas Ajai MSCS 2nd year, interest: Data Engineering
- Alehsan Aliyev MSCS 1st year, interest: Speech Technology, Agentic systems
- Kangyuan Ding MSCE 1st year, interest: Agentic systems, Robotics Learning
- Tianjun Feng MECE 2nd year, interest: Applied AI, Agent benchmarking

1. Objectives

What are you trying to do? Articulate your objectives using absolutely no jargon.

Fix a particular agent workflow, [e.g planner-solver-verifier], find out the best configuration [search space incl: model choice, # of retries*, etc] such that the objective function**[multi objective: cost & success rate. Weighted avg/constrained optimization, pareto frontier search] is optimized. We also consider the cost of finding that best configuration and the quality of that configuration, compare it against Brute Force and random search and other benchmarks/metrics

*retry-aware: when agent fails for the first few times, retry with same or different model. Model choice/strategy is part of the search space.

** : We plan to use single-objective to begin with, then use multi-objective as we are more familiar with retry-aware setups.

1. Objectives

We want to take a fixed agent workflow, such as a **planner**, **solver**, and **verifier**, and find the best way to execute it in terms of success rate and cost.

Different models can be used at different steps, and failed steps can be retried using the same or a different model. We want to find a setup that gives strong results while keeping cost low, and later also consider execution time.

We also care about the cost of finding that setup itself. Our method should reach a good configuration without having to test every possible combination.

We will compare it with simple approaches such as trying everything, random search, and existing methods, looking at both the quality of the final setup and the effort required to find it.

2. Current practice

AgentOpt compares multiple search algorithms for non-retry-aware multi-stage agent pipelines, efficiently identifying model combinations that balance task performance, cost, and latency through a user-defined multi-objective utility rather than exhaustively evaluating every configuration.

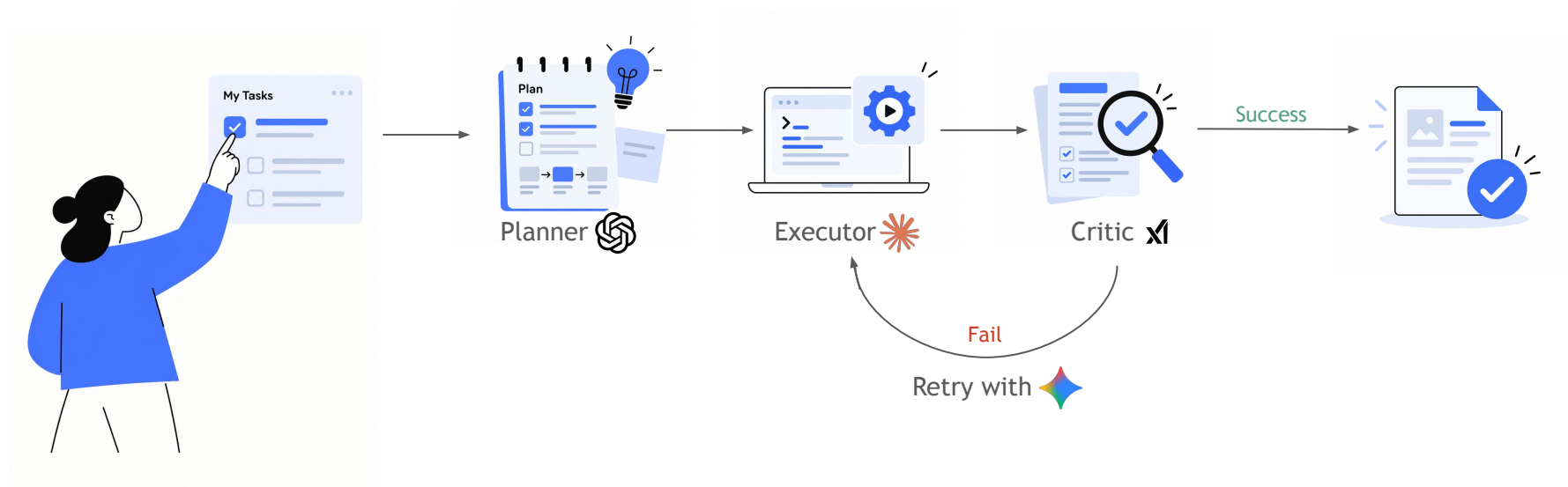
VineLM reduces the cost of optimizing agent workflows by modeling per-invocation model choices as an execution trie and reusing shared prefixes through checkpointing and sparse profiling.

SCOPE reduces the cost of configuring compound AI systems by selectively evaluating promising model configurations and pruning uncompetitive ones while satisfying a target quality constraint.

3. Your approach

- Extend model configuration search to include **retry attempts**, not just the initial workflow stages.
- Allow each retry to use a **different model**, enabling strategies such as cheap-first execution with selective escalation.
- Treat verifier/critic failures as additional **model-selection decision points**.
- This creates a much larger retry-aware configuration space that cannot be exhaustively profiled.
- Reuse shared execution prefixes and focus profiling on promising or uncertain configurations to reduce search cost.
- **Hypothesis:** retry-aware model policies can maintain task success while reducing overall execution cost compared with fixed model assignments.

3. Our approach



Our focus: Retry-aware model configuration

4. Impact

Lower cost per successful task. Cheap models can handle easy cases, while stronger models are used only when retries are actually needed.

Adapt to different business priorities. The same workflow can be configured differently depending on whether the priority is higher quality, lower latency or lower cost.

Easier adaptation when models or prices change. Instead of manually redesigning the workflow, the end users can search for a new effective strategy when new models become available or deployment conditions change.

4. Impact

Lower cost per successful task. Cheap models can handle easy cases, while stronger models are used only when retries are actually needed.

Adapt to different business priorities. The same workflow can be configured differently depending on whether the priority is higher quality, lower latency or lower cost.

Easier adaptation when models or prices change. Instead of manually redesigning the workflow, the end users can search for a new effective strategy when new models become available or deployment conditions change.

5. Risks

The retry part can

- explode the search space, making the optimum hard to find;
- be controlled by another LLM, in order to be context aware (eg. know the reason of failure. Is it big mess-up? Is it just a formatting issue?). Then our retry aware setup don't add much novelty, compared to the problem setup and algorithms without the retry-aware part.

8. Success criteria

Midterm: address the retry aware part of the problem statement, and run similar experiments as the AgentOpt paper, such as investigate on a single objective case.

Final: on top of results of the midterm, address the pareto frontier optimization problem, produce a paper ready for submission, and hopefully do better than naive methods like random search. Be able to make an impact as said in #4.

7: Model Selection for Dynamically Structured Agentic Workflows

Avishek Rao, Laurens Michielsen, Kevin Chuang,
Rushin Bhatt

0. Intros

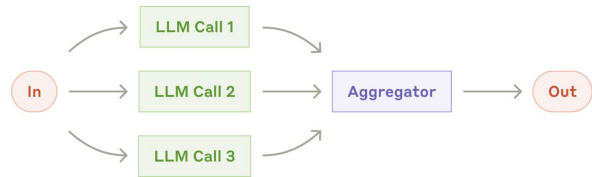
- Avishek (Avi) Rao , MS CS, ML Track, VC investor - AI x Climate Tech
- Laurens Michielsen, MS CS, Agentic AI for data infra, Driverless Race Cars
- Kevin Chuang, MS CS,
- Rushin Bhatt, MS DS, ML Research & Agentic AI, Pianist

1. Objectives

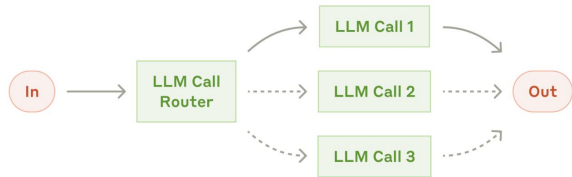
- Agentic applications combine language-model calls with tools to complete **multi-step tasks**
- Choosing a model separately for each invocation allows a system to balance **accuracy, monetary cost, and latency**
- However, the workflow's structure emerges during execution, meaning that a model choice must be made before knowing all the work that will run
- Our objective is to **build a “full workflow aware” model selection system** that not only makes good model choices for a specific task (based on accuracy, cost, and latency), but also takes into account additional tasks that are part of the workflow and are launched in parallel or triggered after a task

1. Objectives (Cont.)

Parallel tasks and fan out



Conditional Routing



2. Current Approaches to Model Routing

- **“Traditional” Approaches:**

- Hard-coded: pre-select models for specific tasks (e.g. summarization, coding)
- Prompt classification: classify difficulty / task type, and then choose a model
- Cascading / Escalation: try a cheap model first, and if confidence is poor, retry with a better / more expensive model

- **Areas of Research:**

- VineLM: Uses a trie of possible model sequences to make fine-grained routing decisions across an agent workflow
- Aragog: Performs just-in-time model routing based on current system load
- Murakkab: Jointly optimizes the workflow configuration, model assignments, and hardware resources used to execute an agentic application.

- **Key existing limitations:**

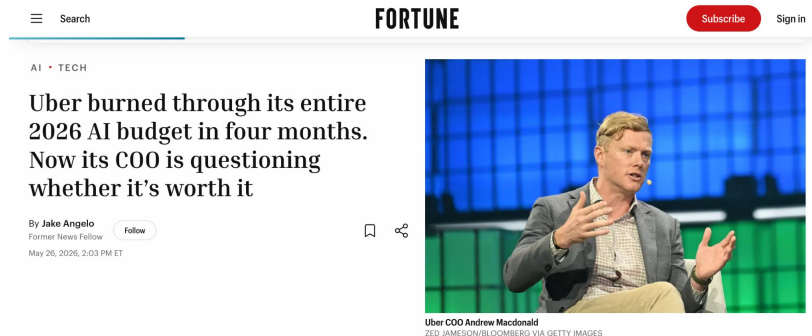
- Current approaches are: 1) not aware of the full agent workflow, leaving room for improvement in model selection, 2) not performing at the right level of granularity - want to choose models per invocation

3. Your approach

- Building on VineLM's trie-based approach to make fine grained routing decisions
 - Context: An execution trie organizes alternative model-choice sequences as a tree, storing shared initial choices once and annotating them with accuracy, cost, and latency estimates
- We will extend that trie into a probabilistic representation that grows as conditional branches and parallel tasks are revealed. Separate nodes will distinguish model choices from execution outcomes, estimating branch probabilities and fanout sizes based on earlier choices and results.
- Potential techniques: low rank matrix approximation; prompt classification
- Hypothesis: creating a “full workflow aware” model selection system will decrease cost at stable accuracy or increase accuracy at stable cost

4. Impact

- **Cost Savings:** easy requests can go to cheaper models, while difficult requests are escalated to frontier models; this helps companies get more work done within their existing AI budgets
- **Lower Latency:** small models are generally faster, so routing simple prompts away from large / reasoning models can reduce average response time
- **Higher Accuracy (at a given budget):** choosing different models for different tasks can beat choosing a single model globally



5. Risks

Anticipated Challenges:

- Scope: agentic workflows can include parallel tasks and conditional branching, and system load could be an important part of the optimization; the various components could lead to multiple papers
- Technical: Feasibility of modeling a complex branching agentic workflow; exponential blowout of tool choices (capture complexity in fanout in the tree); computational overhead of router

Mitigants:

- Scope: we are going to focus on support for conditional branching for this paper
- Technical: the data structure likely won't capture how dynamic a full workflow could be, but could still provide real cost / accuracy benefits; we've identified some techniques (discussed previously) that may help us solve the technical challenges

8. Success criteria

Midterm:

- Execution milestone: Adding support for conditional branching in workflow,
- Baseline: one model throughout a workflow,
- Measurable success criterion: increasing accuracy with stable cost, reducing cost with stable accuracy

Final:

- Execution: Representation of trie for a coding agent that has several tool options,
- Comparison: with results from Murakkab paper,
- Measurable success criterion: same success criterion as above

9: Live Load Balancing for Agentic Workloads Using CXL

Edward Guo, Mani Tofigh, Justin Hou

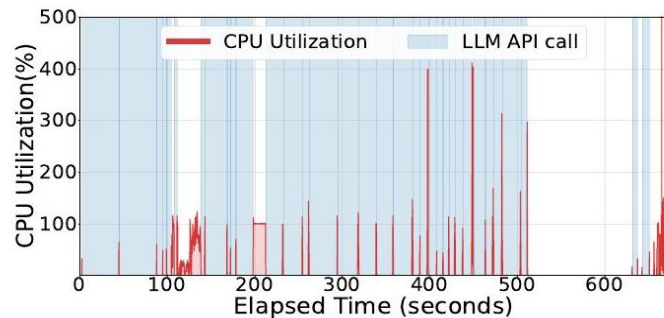
0. Intros

- Edward Guo, PhD, CS - Virtualization, CPU Scheduling, CXL
- Mani Tofigh, PhD, CS – Virtualization, Operating Systems, Comp. Arch
- Justin G Hou, Undergraduate, CS – Distributed systems, Scheduling

1. Objectives

- AI agents spend much of their time waiting, then suddenly need significant compute or memory. **A machine can support many more agents than are active at any one moment.**
- When several wake together, they can overload the machine and hurt everyone's performance. An agent run with even minorly degraded performance could be unsuitable for training. Providers avoid this by provisioning too much capacity, most of which goes unused.

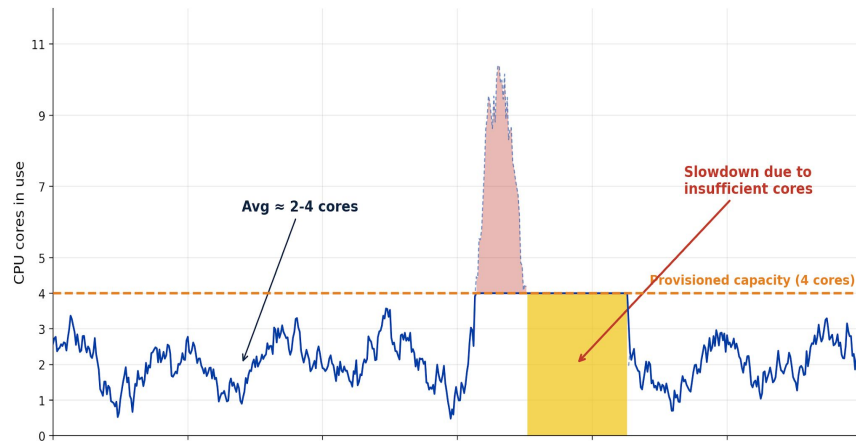
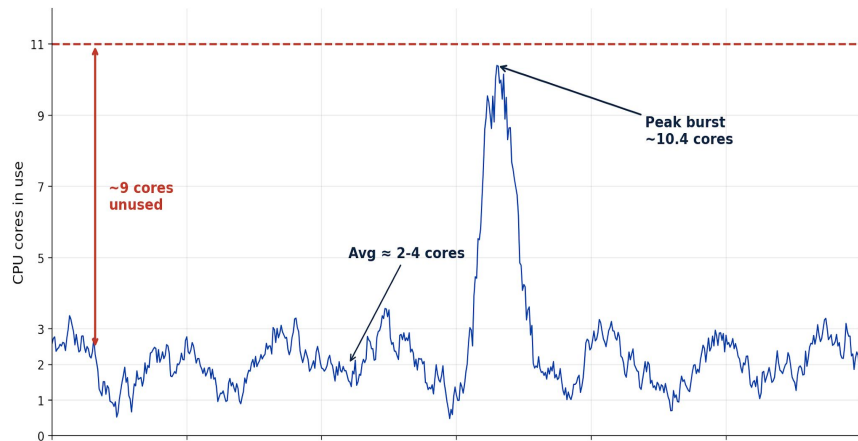
Objective: Pack as many agents onto limited server capacity as possible without degrading performance of the agents.



Overprovision

vs

Underprovision



2. Current practice

- **Today:** Providers substantially overprovision such that servers can handle sudden bursts of work, protecting workloads from severe performance degradation. Even then, sometimes servers are still overloaded, and providers employ different flavours of **live migration**

Post Copy: The task starts running on another server first, while state is streamed in lazily.

Pre Copy: The state is streamed to another server while the task is still running.

Stop-and-Copy: The task is paused, state is moved, then the task is started on the other server.

Post-Copy is too expensive at runtime, Pre-Copy is too slow to react, and **Stop-and-Copy** stops the workload for seconds, sometimes minutes.

The penalty of the migration prevents aggressive load balancing, and therefore prevents aggressive packing.

3.Approach

- **Starting point:** the lab has a **zero-copy** CXL-based mechanism for near-instant migration of agentic containers between servers
 - **Our approach:** intentionally oversubscribe more aggressively; redistribute workloads when simultaneous bursts create contention.
 - **Policy questions:** when should we migrate, which workload should move, and where? Should decisions differ for CPU, memory, network, or other bottlenecks?
 - **Early evidence:** the existing simple, non-aggressive policy shows a promising preliminary improvement. We will test how much further cheap migration lets us push the hardware.
- Hypothesis:** once migration is cheap enough, aggressive provisioning plus rapid repair of unlucky placements beats reserving spare capacity for worst-case bursts.

4. Impact

- **AI infrastructure providers:** serve more concurrent agent workloads and achieve higher utilization on the same hardware, without the traditional tail-latency cost of oversubscription.
- **Users:** keep workloads responsive even when many colocated agents become active at once.
- **Cost:** If fewer machines can serve the same amount of workloads, then we can cut on additional hardware costs.

5. Risks

1. CXL Latency. It is more expensive to access local memory than local DRAM and that penalty might outweigh the benefits

Mitigation: characterize the workloads; potentially do tiering when memory latency becomes a factor.

2.Pool Oversizing. If the task is too small and the server is too large, the server might already do a “good enough” job of absorbing peaks.

Mitigation: Test different workloads, different hardware setups, and different resource bottlenecks.

3. Different resource timescales. CPU, memory, network, and other bottlenecks may favor different migration decisions.

Mitigation: isolate bottlenecks; compare general and resource-specific policies.

6. Success criteria

Midterm

- Set up the CXL testbed and a preliminary scheduling algorithm.
- An automated harness that runs arbitrary workloads while exchanging different scheduling policies.
- Implement one or more migration policies; demonstrate better efficiency or workload density with one or more workloads.

Final

- A breakdown of various workload suites and their performance at higher oversubscription levels.
- Differentiate between workload types, and use that information to reduce latency penalties from CXL.
- Identify safe provisioning limits, where migration helps, and where benefits stop.

12: Latency-and-Cost-Aware Parallel Coding Agents

Amaan Sheikh

Adit Prabhu

Sreeram Kondapalli

Nikhil Chagam

0. Intros

- Amaan Sheikh
 - MSCS 2nd year
 - interests: Multimodal AI and Inference Optimization
- Adit Prabhu
 - MSCS 2nd year
 - interests: Visual Generative Models and Efficient Inference
- Sreeram Kondapalli
 - MSCS 2nd year
 - interests: Enterprise Agents and Systems
- Nikhil Chagam
 - MSCS 2nd year
 - interests: Inference-Time Optimization and Cost-Efficient AI Systems

1. Objectives

What are you trying to do?

- Build a controller that decides how many attempts a coding task gets, while those attempts are still running
- Spend less on easy tasks and more on hard ones, inside a fixed budget and deadline
- Show this beats current practices, which picks one fixed number of attempts for every task

2. Current practice

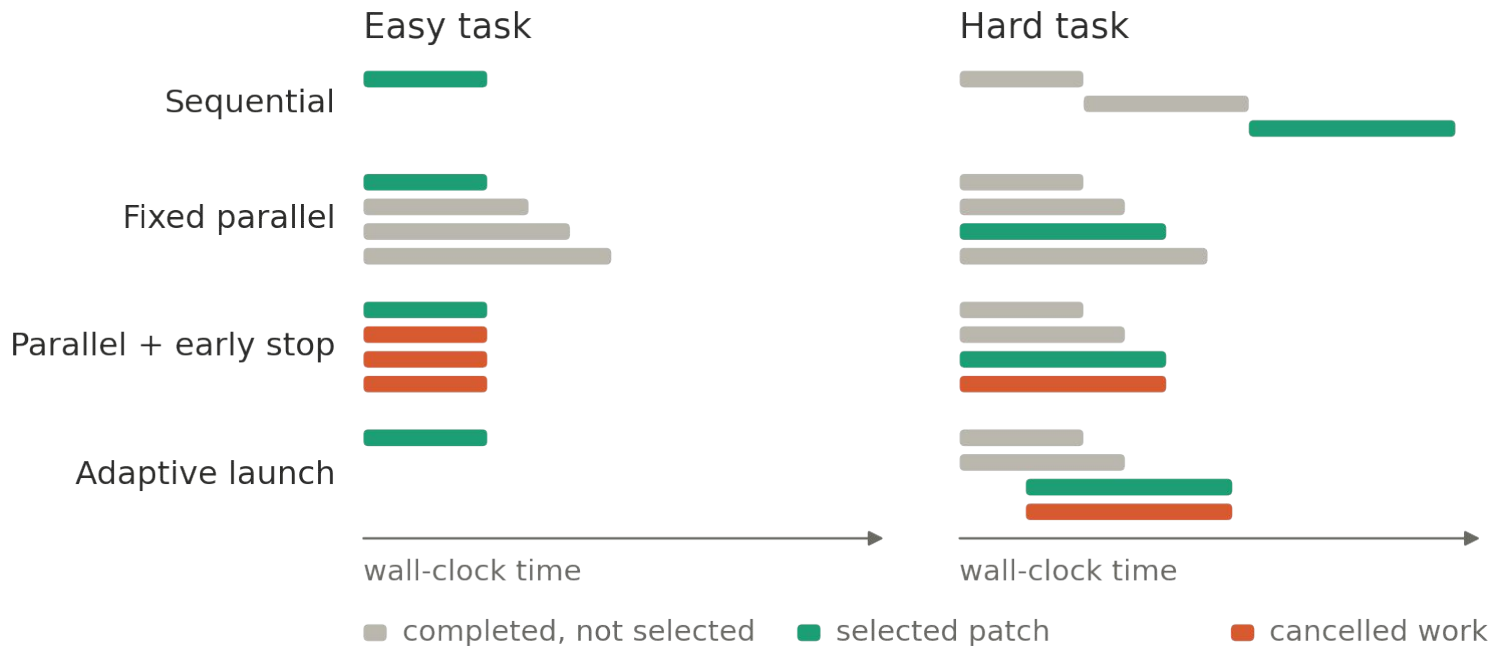
- **Best-of-N:** run N independent attempts, then pick one patch with a selector that never sees the grading tests
- **Sequential expansion:** attempts run back to back and stop at the first acceptable patch - cheap, but latency is the sum of all attempts
- **Fixed parallel best-of-N:** all N launch at once - low latency, but pays for N rollouts even when one would have sufficed
- **AI21 budget-aware execution:** adds early stopping to cascaded and parallel schedules, tracing a cost/latency/quality Pareto frontier
- **Kalayci et al. (2025):** treat each generation as a Pandora's Box and stop adaptively, matching best-of-N with 15-35% fewer generations
- **Limit:** N and the schedule are chosen before any evidence arrives, and the theory assumes instant atomic draws, not minutes-long cancellable rollouts

3. Your approach

- A controller wraps an existing coding agent and each rollout is an independent attempt in an isolated workspace
- At each decision point it picks one action: launch another rollout, wait on the ones running, or select a patch and stop
- **What is new:** attempts launch mid-flight, while earlier rollouts are still running, instead of fixing N up front
- **Signals:** intermediate test results, tool progress, disagreement among candidate patches, and predicted remaining runtime
- Waiting is nearly free when a rollout is close to done, so residual-runtime prediction is what separates this from static best-of-N
- **Hypothesis:** per-task allocation beats any fixed N, because difficulty varies far more across tasks than one global N can absorb
- **Limitation:** gains are capped by selector accuracy, and the controller's own overhead counts against its budget

3. Your approach

Identical rollouts under each policy. Bar width is latency; total bar area is cost.



4. Impact

- Coding agents are moving into CI and code review, where both the token bill and the wait are real operating costs
- **For developers:** lower wait at the same resolve rate, because the search stops the moment a patch clears the bar
- **For providers:** fewer tokens at the same resolve rate, because attempts are not launched on tasks one rollout already solved
- A fixed number of attempts forces one spending decision onto every task; a controller turns that into a per-task budget

5. Risks

- **Correlated failures:** rollouts from one model fail alike, so coverage saturates and extra attempts add nothing
 - **Mitigation:** measure the coverage curve first; vary temperature, prompt etc to buy diversity
- **Right patch, wrong pick:** a correct patch is among the candidates and we submit a different one
 - **Mitigation:** report what the resolve rate would be with perfect selection, beside every result
- **Signals that lie:** deleting the failing test also makes the suite pass, and a stuck agent still looks busy
 - **Mitigation:** combine independent signals, keep grading tests hidden, and hand-audit accepted patches
- **Overhead and variance:** controller cost can be high, and agent runs are noisy enough to hide small effects
 - **Mitigation:** charge every token including cancelled work; run multiple seeds and report confidence intervals

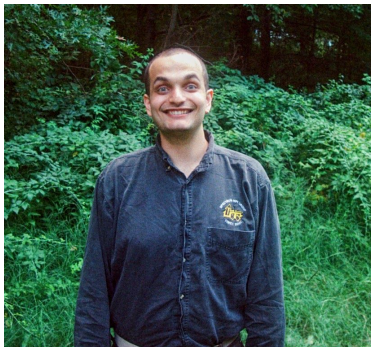
6. Success criteria

- **Midterm:** a controller harness running isolated rollouts with cancellation, plus a patch selector and full cost/latency accounting
- **Midterm criterion:** reproduce fixed parallel best-of-N and sequential expansion as baselines on a held-out SWE-bench Verified slice
- **Final:** compare fixed best-of-N, sequential expansion, parallel early stopping, and adaptive launch on identical tasks and seeds
- **Final criterion:** at matched resolve rate, adaptive cuts cost or p90 latency by more than the seed-to-seed variance
- Grading tests stay hidden from the controller and the selector throughout
- **Metrics:** resolved within budget and deadline; tokens including cancelled work; p50/p90 latency; selector accuracy vs. oracle; controller overhead

13: Benchmarking Web-Agent Performance and Interference

Namwar Ahmad Rauf, Harry Minsky

0. Intros



Harry Minsky, MS CS
Networking and systems
with a focus on multi-path
transport layer protocols

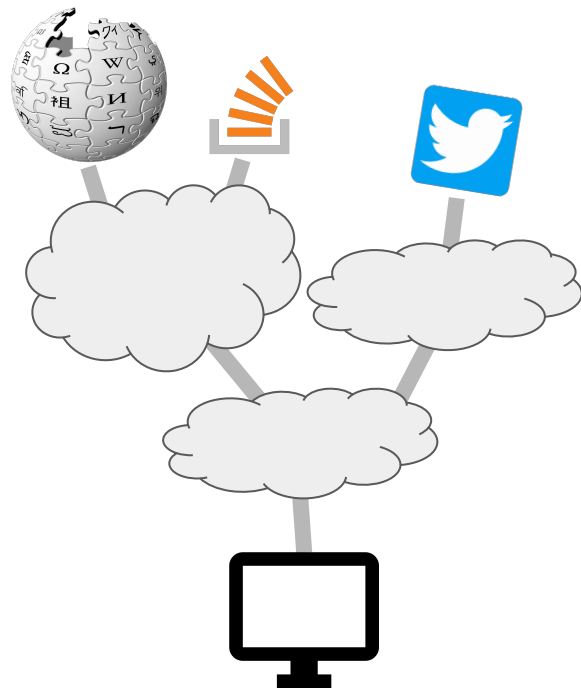


Namwar Ahmad Rauf,
1st Yr PhD CS
Network measurement

Internet routing, network traffic,
network modelling

1. Objectives

- When a user asks an AI to “look up XYZ”, we want to determine how much more **work** does that **website** have to do than if the person looked it up themselves?
- Our aim: measure what one person’s question costs a website when an AI answers it instead of a user themselves.
 - What happens to the site?
 - What happens to others also using the site when this scales?



2. Current practice

Industry work:

- Wikimedia, Cloudflare, etc. report what fraction of traffic is by automated agents, but don't have a metric for how well users (agents and humans) are served by their platforms.

Academic work:

- [Rethinking Web Cache Design for the AI Era, SoCC'25 \(ETH Zurich\)](#)
 - Measured impact of growing AI traffic on Wikipedia clone and found that share of requests that have to be done from scratch (cache miss) nearly doubles when AI is a quarter of the total traffic
- [A Worldwide View of Browsing the World Wide Web, ICM' 22 \(Google, Stanford\)](#)
 - Quantitative review of human browsing habits conducted by google researcher, found that "human browsing is heavily concentrated on top sites...Top websites receive several orders of magnitude more traffic than lower ranked sites"

Limits:

- Their AI visitor program was a depth-first program (walks through sites visiting every link).
- Does not represent behavior of an AI assistant answering a user's queries.
- Don't study behavior of different types of agents

3. The approach

Testbed:

- Real website software on a server (primarily Wikipedia via WebArena) + standard cache and rate limiter in front
- Every in and out request is recorded and timestamped

Question Design:

- Catalog of qs: “cost of product X”, “author of article Y”, etc
- Which item to ask about constructed from Wikipedia popularity data (not arbitrarily decided)
- Same question to every visitor, so difference being measured is across visitors

Visitor Types: Browser Agent (Browser Use), Structure Agent (Playwright MCP),
Text-fetch agent

- Each type x {cheap model, frontier model} -> to see how the model choice impacts time measured

3.2 The approach

- Per question we log: requests, bytes, distinct URLs touched, whether re-downloaded content, model thinking time vs website-waiting time, correctness of answer retrieved

Human Baseline: model the human behavior

- Real Wikipedia pageview data (human identified traffic) -> popularity curve from measured data
- Follow real page-to-page flow from Wikipedia clickstream data of real users
- Cost of each page (HTML + image, css, etc). Browser caches on repeat visits by users (HTTP Archive)

Hypotheses:

- H1: text-only agent worst for cache (wandering more) -> more load on sites
- H2: Smarter cache eviction policies might not be needed -> simply prioritize human user cache access (keep AI visits out of cache)

4. Impact

A measurement-backed proposal for changes in cloud providers

- Small sites, nonprofits, and documentation sites see slow down from overuse. People not using AI assistants experience slow pages and even wrongful blocking by anti-bot defenses. A more explicit model for agents to identify themselves and their needs could benefit both agents and site hosts.
- Cloud providers who host websites would stand to benefit from understanding how agents access websites and how to efficiently serve (or not serve) them.
- Users of web agents may be able to reduce the amount of usage they require from the model to solve their problems.

5. Risks

- Observing web usage from a closed model poses a range of information accessibility issues
 - Mitigation: controlled web environments with simulated websites, containing agents to using tools we can monitor, open source/locally hosted models
- Human web usage can be hard to characterize, and a user study would be out of scope
 - Mitigation: Investigate existing user studies, model our web agent benchmarks around human data that already exists
- Web agents have thus far operated on the assumption that the web is human-generated content. As agent use increases, general usefulness may not scale well
 - Mitigation: As a measurement-based study, we are looking at the state of current infrastructure to provide a foundation for future work.

6. Success criteria

- Midterm: Design a testbed for measuring agentic web use patterns, and procure a baseline human use pattern to compare it to
 - Be able to sufficiently monitor network usage of web agents (simulated sites, observable web-browsing tools that we can have agents use)
- Final: Implemented testbed. Collected a substantial measurement of web agent traffic use, and derive some meaningful insight from the measurements
 - Identify key patterns (or lack thereof) in typical web agent network usage that would be relevant to site operators trying to better serve (or identify) agents
- Metrics:
 - Sites visited: Site diversity, correlation between sites, number of sites
 - HTTP requests: request success rate, # of requests
 - Fulfillment of tasks given

16: Efficient Analysis of Agent Execution Histories

Dean Sysman, Petra Hedesiu, Amina Isayeva, Shawn Liu, Yilin Ye

0. Intros

- **Dean Sysman**, 1st year EE PhD. Founder of big data Cyber company, and finalizing first research project improving performance of Linux cache using AI, under Prof Asaf Cidon and Kostis which will be submitted to MLSys27.
- **Petra Hedesiu**, 2nd-year undergraduate in SEAS. First-authored MLSys security paper under Prof. Asaf Cidon and worked with Dean on the Linux cache project
- **Amina Isayeva**, 1st year MSCS student in the Machine Learning pathway, working at Columbia's Speech Lab and full-time as a software engineer in AI and safety. I'm interested in building reliable AI agents that safely automate real-world workflows.
- **Shawn Liu**, 1st-year MSCS (moving to the thesis track). I have some experience in LLM agent evaluation suites.
- **Yilin Ye**, first-year MS CS student in the Thesis Pathway, interested in AI agents and currently exploring agent evaluation.

1. Objectives

AI agents generate large, messy execution histories spanning model interactions, tool calls, application logs, database operations, and changes to files, processes, and network state.

These records could reveal how agents approach tasks, where failures originate, whether expected procedures are followed, and which behaviors predict progress, success, or unsafe outcomes. The broader goal is to make these histories useful for understanding and improving agent behavior.

2. Current practice

- [TrajectoryDB](#): A New Database for Agent Trajectories— the closest systems vision, especially its treatment of trajectories as a data type combining hierarchy, text, temporal order, and lineage
- [Arming Data Agents with Tribal Knowledge](#)— demonstrates how experience extracted from prior failures can improve future agent behavior.
- [Agentic Data Environments](#)— motivates branching, cross-component state management, provenance, and data-flow control.
- [ATLAS: Discovering Agent Strategies through LLM-Guided Abstraction and Automata Learning](#)
- [TRAIL](#): Pasting entire log into long-context LLMs: only 11% accuracy
- [Docent](#): uses LLMs to summarize, cluster, and search agent transcripts
- [TraceGraph](#): maps agent runs into one graph to show where agents succeed, get stuck and recover

3. Your approach

1. Agents performing tasks from Terminal-Bench, using StateFork and Waypoint to capture branchable environment state alongside agent trajectories.
2. Replicate TrajectoryDB framework to capture agent trajectories.
3. Reconcile branchable environment states and agent trajectories.
4. Assemble a corpus containing multiple successful and unsuccessful runs and define a benchmark of representative analysis questions.
5. Examine recurring strategies and action sequences, where successful and failed runs diverge, whether agents conform to an expected process, which actions cause consequential environment changes, and how early success, failure, or a safety violation can be detected.
6. Propose and evaluate a system for answering these questions against existing ad hoc workflows.

4. Impact

- The semester objective is to answer them more quickly or with substantially less analyst effort while preserving analysis accuracy and provenance.
- Lead to better outcomes in cost, speed and accuracy
- Additional new functional abilities can potentially lead to new capabilities - Historical analysis, rollback, security monitoring, etc
- **Who cares?**
 - Agent developers debugging failures.
 - Evaluation and safety teams auditing whether agents followed procedures or caused unsafe changes.
 - Researchers mining trajectories for training data, memory, and cost optimization.
 - Regulators across industries interested in answers to questions like “Did your agent touch my data and if so what did it do with it?” (finance, consulting)

5. Risks

1. **Prediction error:** Insights from analysed data do not initially infer benefits
 - Mitigation: Understanding impact of prior work, narrowing down and expanding use cases
2. **Underlying platform dependency:** Waypoint/StateFork or others that do not enable the desired use cases
 - Mitigation: Aligning use cases to benefits and existing capabilities, developing custom capabilities
3. **Model/Agent variability:** Differences in Models/Agents and frameworks lead to variance in outcomes
 - Mitigation: Prove out outcomes in a strategic approach based on common singular cases
4. **Dilution of effort across use cases:** Potential for multiple benefits might make project be too spread out
 - Mitigation: Focus on highest likelihood successful directions before branching out

8. Success criteria

- **Midterm:** Robust ability for benchmarking and analysing agent runs
 - Benchmark of Agent Branching
- **Final:** Newly discovered ability as a result of midterm ability, demonstrated in one or more of the metrics
- **Metrics:**
 - Improvements in token usage or avoiding predetermined unsuccessful Agent branches
 - Reduced cost on same result
 - Benchmark accuracy of decision paths of agents
 - Monitor/Prevent agent undesired actions paths
 - Undo/Rollback/Snapshot on agent capability